Parameter Golf — Hypothesis #001

Gated relu² MLP for quantization-friendly weight distributions

2026-03-22 · Target: move val_bpb by hundredths, not thousandths

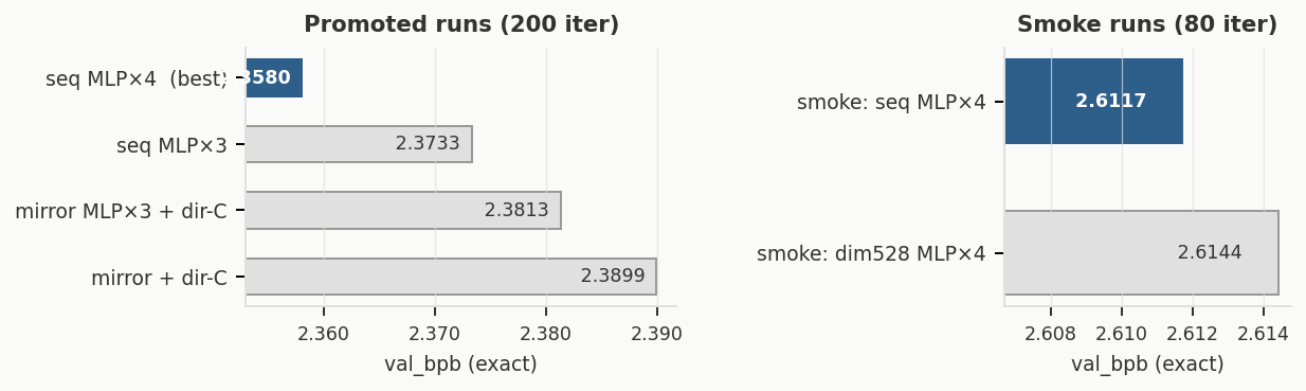
---

1. What the evidence says

Four facts from recent experiments, in order of importance:

- ✔ MLP capacity is the binding constraint — MLP×4 beat MLP×3 by 0.015 bpb on promoted runs
- ✘ Width doesn't help — dim 528 lost to dim 512 despite more raw parameters
- ✘ Shared-core recurrence / stabilization consistently hurt post-compression score
- ■ Control tensors (fp16 passthrough) are a direct byte cost with diminishing returns

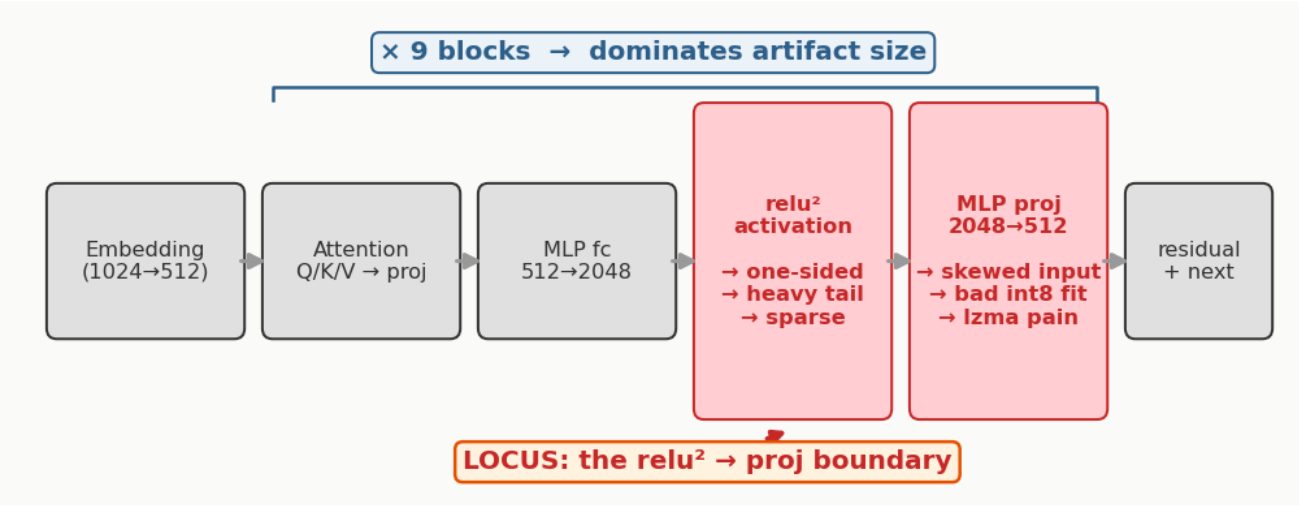
The model is not undertrained or under-dimensioned. It is using its dimensions inefficiently through the quantization bottleneck. The MLP is where most parameters live, and the activation function determines how those parameters compress.

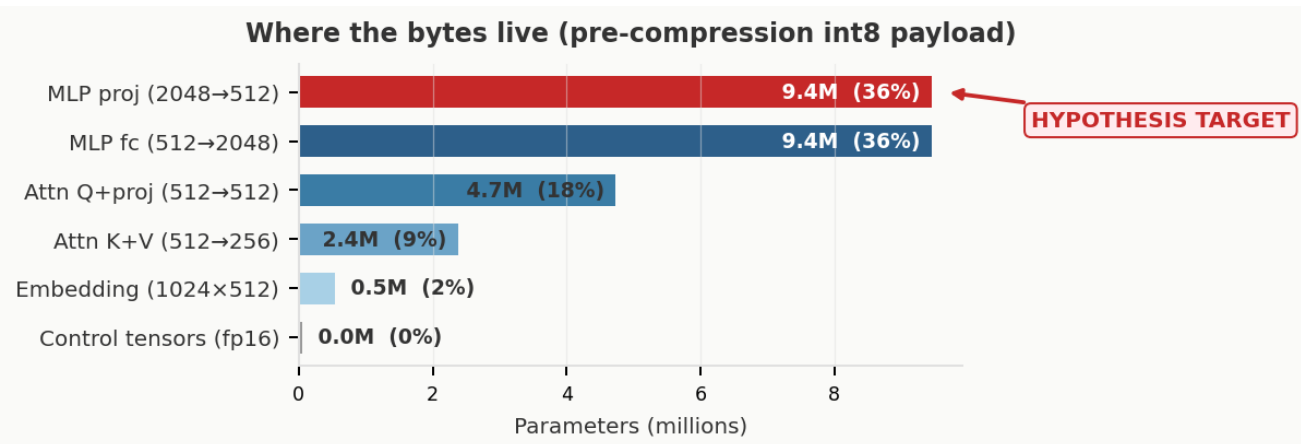


2. Locus — where is the failure?

The MLP projection matrix (`mlp.proj`) in each of the 9 blocks. These 9 matrices hold **9.4M parameters** — roughly 36% of all model parameters — and they receive the output of relu², which is strictly non-negative and heavy-tailed.

This is a **boundary problem**: the failure occurs at the boundary between the activation function and the projection that must survive int8 quantization + lzma compression.





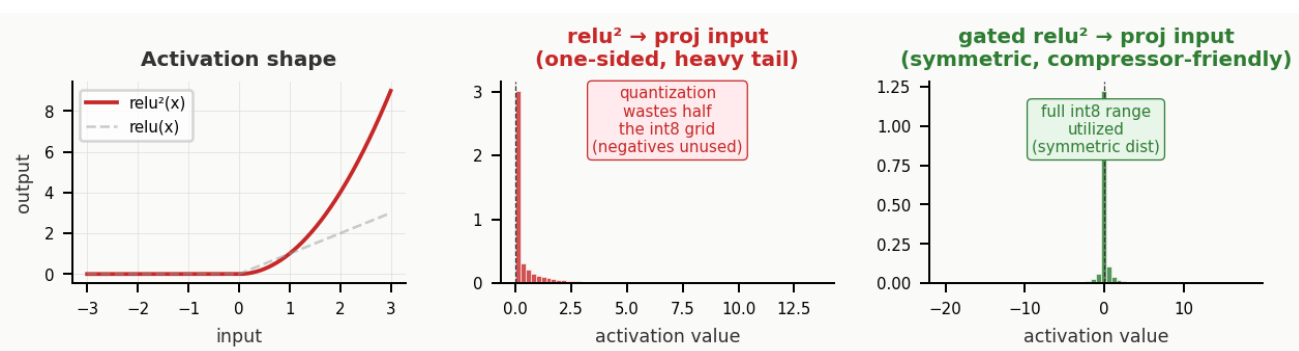
3. Mechanism — what could fix it?

The core argument in three steps:

Step A: The input distribution to mlp.proj is pathological for int8. relu² outputs are ≥ 0 and heavy-tailed. The int8 grid spans [–127, +127], but negative values are never used. Half the representable states are wasted. The heavy tail forces high per-row scales, crushing precision for typical activations.

Step B: A gated activation produces symmetric, lighter-tailed inputs to proj. gate(x) · up(x) multiplies the one-sided relu² gate output by a signed linear projection. The result is symmetric around zero and uses the full int8 range.

Step C: Symmetric distributions compress better under lzma. lzma exploits byte-level patterns. Symmetric int8 values around zero have more regular byte patterns than one-sided values, improving the compression ratio for the same information content.



Why not standard SwiGLU?

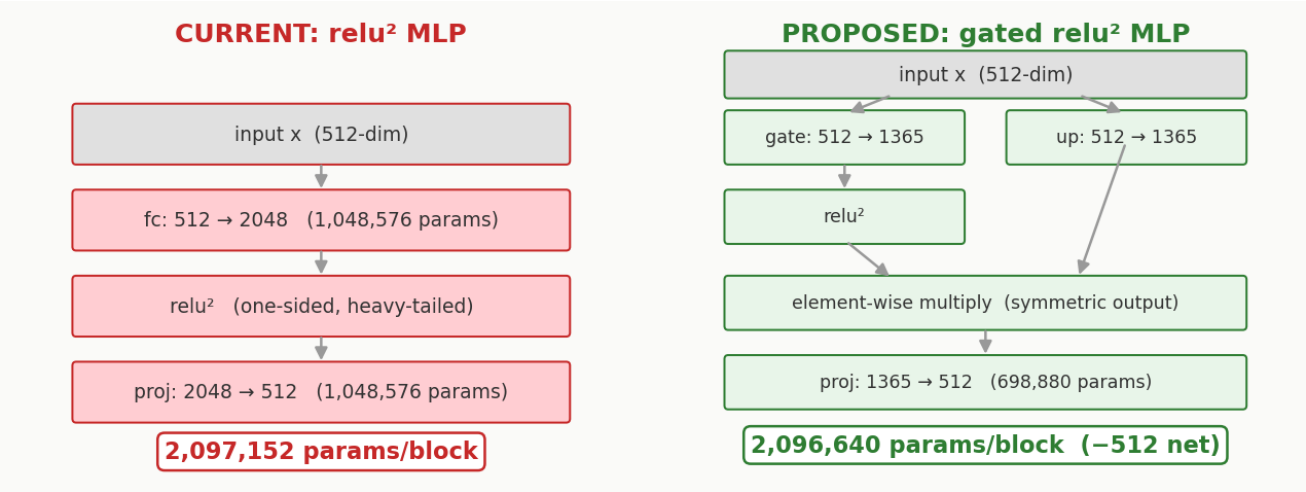
SwiGLU uses SiLU (swish) as the gate activation. That would work, but relu² has a known advantage in this codebase: it is cheaper to compute and the gradient signal is compatible with the Muon optimizer. The hypothesis is specifically about **gating with relu²**, not about changing the activation family. The key insight is that the gate × up product creates symmetry *regardless of the gate activation shape*.

4. The patch — minimal code change

Replace the MLP class. One class, ~10 lines changed:

```
class MLP(nn.Module):
  def __init__(self, dim: int, mlp_mult: int):
    super().__init__()
    hidden = int(dim * mlp_mult * 2 / 3)  # 1365 for dim=512, mult=4
    self.fc_gate = CastedLinear(dim, hidden)
    self.fc_up   = CastedLinear(dim, hidden)
    self.proj    = CastedLinear(hidden, dim)

  def __call__(self, x: mx.array) -> mx.array:
    return self.proj(nn.relu2(self.fc_gate(x)) * self.fc_up(x))
```



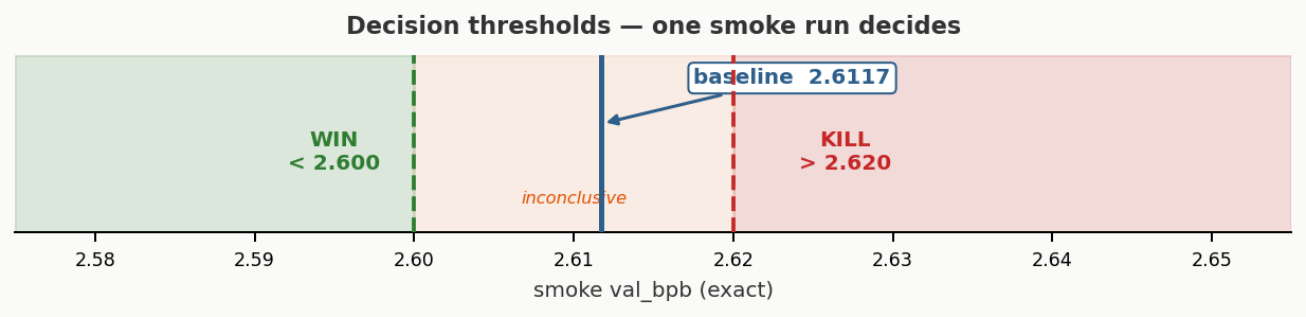
5. Smoke run protocol

One run. 80 iterations on smoke data. Compare exact post-quantized val_bpb against the smoke baseline.

Parameter	Value	Rationale
MLP activation	gated relu²	The hypothesis under test
MLP hidden dim	1365	int(512 × 4 × 2/3) — param neutral
MODEL_DIM	512	Unchanged
NUM_LAYERS	9	Unchanged
MLP_MULT	4	Unchanged (reinterpreted for gating)
ITERATIONS	80	Smoke length

Dataset	fineweb10B_sp1024_smoke	Standard smoke data
Compressor	lzma	Match baseline compressor
Everything else	identical to baseline	Isolate the variable

6. Decision criteria — exact thresholds



Outcome	Smoke val_bpb	Action
WIN	< 2.600	Promote to 200-iter run immediately
INTERESTING	2.600 – 2.611	Worth a promoted run to confirm
NOISE	2.611 – 2.620	Within noise of baseline — inconclusive, park it
KILL	> 2.620	Kill the hypothesis. Do not tune. Move on.

7. Pre-flight diagnostic (optional but recommended)

Before running the smoke, load the best `.ptx` artifact and dump per-row scale statistics for every quantized tensor. If `mlp.proj` rows show high scale variance relative to `mlp.fc` and attention matrices, that confirms the mechanistic premise. If they don't, reconsider the locus before burning a smoke run.

20-line script: load artifact → extract `scales` dict → compute `std(scales) / mean(scales)` per tensor → rank by CV. The hypothesis predicts `mlp.proj` will have the highest coefficient of variation.

---

8. What this hypothesis is NOT

✘ Not a global capacity change — param count is held constant within ±0.02%

✘ Not a training trick — no LR change, no schedule change, no regularizer

✘ Not a thousandths shave — the mechanism targets the largest tensor family and changes the distribution shape, which can move by hundredths

✘ Not another control tensor — zero new fp16 passthrough cost

✘ Not speculative — the premise (one-sided inputs quantize worse) is directly verifiable from the existing artifact before running anything

---